Universidad de Manizales

Facultad de ciencias e ingeniería

Construcción de un Sistema de Gestión Académica utilizando Programación

Actividad Colaborativa N°2: Evolución del Sistema de Gestión Académica

Jonathan Gómez Salazar

Omar Picón Meza

Natalia Marcela Castellanos Gómez

28 de oct. de 2026

Introducción

Este documento explica y justifica las principales decisiones de diseño tomadas por el equipo al evolucionar el Sistema de Gestión Académica. En la Actividad Colaborativa N°1 se construyó una primera versión en Python que representaba personas (estudiantes, docentes y administrativos), programas académicos y asignaturas, aplicando clases, constructores, encapsulamiento, herencia, sobre escritura y polimorfismo.

En la Actividad Colaborativa N°2 la universidad necesita incorporar la oferta de grupos académicos y la matrícula de estudiantes. La actividad nos permitió realizar el trabajo sobre el mismo proyecto y el mismo repositorio, sin crear un sistema nuevo. La versión de la AC1 permanece identificable en el repositorio mediante su etiqueta y no fue modificada.

La evolución se apoyó en tres conceptos nuevos: las clases abstractas (para generalizar a las personas), la composición (para relacionar un grupo con su horario) y las excepciones personalizadas (para controlar situaciones anormales durante la matrícula y las búsquedas). A lo largo del documento se hace referencia directa a las clases y métodos implementados.

Descripción general de la solución

La solución mantiene todas las clases de la AC1 y agrega las necesarias para la oferta académica y el control de errores. El archivo principal (main.py) crea los objetos, ejecuta las operaciones de matrícula y demuestra cada excepción sin que el programa se detenga.

Clase	Responsabilidad	Novedad en la AC2
Persona	Datos comunes (identificación, nombre, correo) y mostrar_informacion( )	Ahora es **abstracta**
Estudiante	Código estudiantil, programa, semestre y promedio	Sigue implementando realizar_actividad_principal( )
Docente	Número de empleado, facultad,	Sigue implementando realizar_actividad_principal( )

	tipo de contratación y horas semanales	
Administrativo	Número de empleado, dependencia, cargo y jornada	Sigue implementando realizar_actividad_principal( )
ProgramaAcademico	Código, nombre, facultad y número de semestres	Sin cambios de estructura
Asignatura	Código, nombre, créditos y programa al que pertenece	Se ofrece en uno o varios grupos
Horario	Día, hora de inicio, hora de finalización y salón	**Nueva**
GrupoAcademico	Código, asignatura, docente, capacidad, estudiantes matriculados y horario	**Nueva**
Excepciones personalizadas	Controlan las situaciones anormales de matrícula y búsqueda	**Nuevas**

El flujo general es el siguiente: se crean programas y asignaturas; se crean docentes y estudiantes; se define un Horario por cada grupo; se crean al menos dos objetos GrupoAcademico que reciben la asignatura, el docente y el horario ya construidos; y finalmente se matriculan y retiran estudiantes, capturando en el programa principal cada excepción que se presente.

Desarrollo

Punto de partida: lo construido en la Actividad Colaborativa N°1

La AC1 dejó tres decisiones que la AC2 respeta y aprovecha. La primera es el encapsulamiento: los atributos de Persona, ProgramaAcademico y Asignatura son privados y se consultan o modifican mediante métodos que validan los valores (el correo no puede quedar vacío, y los semestres de un programa y los créditos de una asignatura deben ser mayores que cero). La segunda es la herencia: Estudiante, Docente y Administrativo heredan de Persona los datos comunes, evitando repetirlos. La tercera es el polimorfismo: todas las personas responden a mostrar_informacion() y a realizar_actividad_principal(), y el programa principal las procesa en un único ciclo sin preguntar por el tipo de cada objeto. Dado que el enunciado prohíbe reemplazar el proyecto, estas decisiones se conservaron y la AC2 se construyó encima de ellas.

Persona como clase abstracta

En el contexto de la universidad no tiene sentido registrar una persona sin saber si es estudiante, docente o administrativo: toda persona vinculada cumple uno de esos tres roles. Por eso la clase Persona, que en la AC1 podía instanciarse, se convirtió en una clase abstracta usando el módulo ABC de Python.

```
from abc import ABC, abstractmethod

class Persona(ABC):
    def __init__(self, identificacion, nombre,
correo):
        self.__identificacion = identificacion
        self.__nombre = nombre
        self.__correo = correo

    def mostrar_informacion(self):
        return f'Identificación:
{self.__identificacion} | Nombre: {self.__nombre}
| Correo: {self.__correo}'

    @abstractmethod
    def realizar_actividad_principal(self):
        pass
```

Esta decisión tiene dos efectos. Primero, el sistema impide crear directamente un objeto Persona: al intentar Persona ('1', 'Ana', 'ana@correo.com') Python lanza un TypeError, y el programa principal lo demuestra capturándolo. Segundo, Persona queda como el lugar donde se concentra lo común (los datos generales y mostrar_informacion()), mientras que lo que solo puede conocerse en cada rol, el

método realizar_actividad_principal(), se declara abstracto y obliga a las clases hijas a definirlo.

Se mantuvo en Persona la implementación concreta de mostrar_informacion() porque su parte general es igual para todos; las clases hijas la sobrescriben para agregar sus datos particulares, reutilizando la versión de la clase padre mediante super(). Así se evita repetir código y se conserva la sobreescritura exigida en la AC1.

Implementación del método abstracto en las clases derivadas

Como realizar_actividad_principal() es abstracto, Estudiante, Docente y Administrativo están obligados a implementarlo; si alguna no lo hiciera, tampoco podría instanciarse. Las tres clases conservan exactamente el mismo nombre del método, pero cada una aporta un comportamiento propio con este:

- **Estudiante:** Informa qué programa académico cursa y en qué semestre se encuentra, usando su atributo programa (un objeto ProgramaAcademico) y su semestre.
- **Docente:** Informa en qué facultad imparte clases, a partir de su atributo facultad.
- **Administrativo:** Informa qué cargo desempeña, en qué cargo y en qué dependencia trabaja.

```
class Estudiante(Persona):
    def realizar_actividad_principal(self):
        return f'{self.nombre} cursa
{self.programa.nombre}, semestre
{self.semestre}.'

class Docente(Persona):
    def realizar_actividad_principal(self):
        return f'{self.nombre} orienta clases en
la facultad de {self.facultad}.'

class Administrativo(Persona):
    def realizar_actividad_principal(self):
        return f'{self.nombre} es {self.cargo} en
{self.dependencia}.'
```

La decisión de dejar el comportamiento en cada clase hija, y no en Persona, responde a que ese comportamiento depende del rol y no puede definirse de forma general sin recurrir a condicionales por tipo, cosa que la actividad prohíbe expresamente.

Organización de la oferta académica: cómo se relacionan las clases

Una asignatura puede ofrecerse en varios grupos, por lo que el grupo se modeló como una clase independiente, GrupoAcademico, y no como un atributo de Asignatura. Un grupo representa una oferta concreta (con su docente, su cupo y su horario), mientras que la asignatura describe qué se enseña.

Las relaciones entre las clases son las siguientes:

- **GrupoAcademico y Asignatura (asociación):** El constructor del grupo recibe un objeto Asignatura ya creado. No se guarda solo el nombre o el código como texto, de modo que desde el grupo se puede acceder a toda la información de la asignatura (créditos, programa, etc.). Una misma Asignatura puede estar asociada a varios grupos.
- **GrupoAcademico y Docente (asociación):** El grupo recibe un objeto Docente ya creado como responsable. Un mismo docente puede tener a cargo varios grupos, y el docente existe, aunque se retire de un grupo.
- **GrupoAcademico y Estudiante (agregación mediante una lista):** El grupo mantiene una lista de objetos Estudiante matriculados. Los estudiantes se crean fuera del grupo y existen independientemente de él; el grupo solo los referencia.
- **GrupoAcademico y Horario (composición):** El grupo contiene un objeto Horario que se explica a continuación.

```
class GrupoAcademico:
    def __init__(self, codigo, asignatura,
docente, capacidad_maxima, horario):
        self.__codigo = codigo
        self.__asignatura = asignatura
objeto Asignatura
        self.__docente = docente
objeto Docente
        self.__capacidad_maxima =
capacidad_maxima
        self.__estudiantes = []
lista de objetos Estudiante
        self.__horario = horario
objeto Horario
```

El Horario (día, hora de inicio, hora de finalización y salón) se representa como un objeto y no como una cadena de texto, porque así cada dato puede consultarse,

validarse y modificarse por separado, y porque deja evidente la relación entre las clases.

La relación entre GrupoAcademico y Horario es de composición por las siguientes razones:

- El horario describe cuándo y dónde se desarrolla un grupo concreto. Un horario no tiene sentido por sí solo, sino como parte de la organización de ese grupo.
- El horario se crea como parte del grupo y su ciclo de vida depende de él: si el grupo desaparece, el horario deja de tener razón de ser.
- El grupo tiene un horario; no es un horario.

No corresponde a herencia porque la herencia expresa una relación de tipo “es un”, de generalización y especialización, como la que existe entre Persona y Estudiante, donde un estudiante es una persona y por eso hereda sus datos y comportamientos. Un GrupoAcademico no es una especialización de Horario ni al revés: no comparten una identidad común ni se sustituyen entre sí. Usar una herencia aquí forzaría a que el grupo heredara atributos como el día o el salón, que no son parte de lo que un grupo es, sino de lo que un grupo tiene.

En términos del diseño, la composición también facilita el mantenimiento: si en el futuro el horario necesita más datos (por ejemplo, la modalidad), el cambio se hace en Horario sin tocar la jerarquía de Persona ni el resto del grupo.

Excepciones personalizadas

La Fase 4 pedía que las situaciones excepcionales se controlaran con excepciones propias y no solo con condicionales que muestren mensajes. Por eso las cinco excepciones se definieron como clases en exceptions/excepciones.py, todas heredando de Exception. Cada una recibe como parámetros los datos que describen la situación (códigos de estudiante, grupo, asignatura o programa) y construye con ellos un mensaje comprensible. Se lanzan con raise desde el método que detecta el problema y se capturan en main.py con try/except, de modo que el programa continúa después de cada error.

Excepción	Situación que controla	Dónde se genera	Dónde se captura
EstudianteYaMatriculadoError	Matricular dos veces al mismo estudiante en un grupo	GrupoAcademico.Matricular_estudiante()	main.py

Excepción	Situación que controla	Dónde se genera	Dónde se captura
GrupoLlenoError	Matricular cuando el grupo ya alcanzó su capacidad máxima	GrupoAcademico.Matricular_estudiante()	main.py
EstudianteNoMatriculadoError	Retirar a un estudiante que no está en el grupo	GrupoAcademico.Retirar_estudiante()	main.py
AsignaturaNoEncontradaError	Buscar una asignatura con un código que no existe	RegistroAcademico.BuscarAsignatura()	main.py
ProgramaNoEncontradoError	Buscar un programa académico con un código que no existe	RegistroAcademico.BuscarPrograma()	main.py

En Matricular_estudiante() se evalúan primero la duplicidad y después la capacidad, y en ambos casos la excepción interrumpe el método antes de agregar al estudiante a la lista, así que el grupo nunca queda en un estado inconsistente. En main.py las llamadas a este método se agrupan en un try que captura ambas excepciones, mientras que Retirar_estudiante() y las búsquedas tienen su propio try/except.

Las dos excepciones de búsqueda necesitaban un lugar donde buscar. Ni asignaturas ni Programa_academico conocen el catálogo completo, y poner esa lógica en el main habría mezclado la prueba con la regla. Por eso se creó la clase RegistroAcademico en controllers/, que guarda una lista de programas y otra de asignaturas, valida con AgregarPrograma() y AgregarAsignatura() que se registren objetos del tipo correcto, y ofrece BuscarPrograma(codigo) y BuscarAsignatura(codigo): cada una recorre su lista y, si termina sin encontrar el código, lanza la excepción correspondiente.

Se conservaron, sin convertirlos en excepciones, los mensajes de los setters (por ejemplo, un correo sin arroba o créditos menores que uno) y la validación de tipo en GrupoAcademico (pasar un texto como asignatura). Son validaciones de datos, que ya funcionaban en AC1 informando el error y sin aplicar el cambio, y se mantuvieron para no alterar el comportamiento ya probado. Las excepciones se reservaron para situaciones del proceso de matrícula y de búsqueda.

Continuidad del polimorfismo de la Actividad Colaborativa N°1

Convertir Persona en clase abstracta no elimina el polimorfismo; al contrario, lo fortalece. Aunque ya no se pueden crear objetos Persona, sí se pueden crear referencias genéricas a ella: la lista del programa principal sigue conteniendo estudiantes, docentes y administrativos mezclados, y se recorre con un único ciclo:

```python
personas = [estudiante_1, estudiante_2, docente_1,
docente_2, administrativo_1, administrativo_2]

for persona in personas:
    print(persona.mostrar_informacion())
    print(persona.realizar_actividad_principal())
```

Python ejecuta automáticamente la versión del método que corresponde al tipo real del objeto, sin condicionales que pregunten si es estudiante, docente o administrativo, sin ciclos separados y sin cambiar el nombre del método. La clase abstracta aporta además una garantía que antes no existía; todo objeto que pueda estar en esa lista debe tener implementado realizar_actividad_principal(), porque de lo contrario no habría podido crearse. El polimorfismo pasa de ser una convención a ser parte obligatoria por el diseño.

Este mismo principio se aprovecha en la nueva funcionalidad: GrupoAcademico recibe un Docente y una lista de Estudiante, y puede mostrar sus datos usando mostrar_informacion() sin conocer los detalles internos de cada clase.

Pruebas realizadas

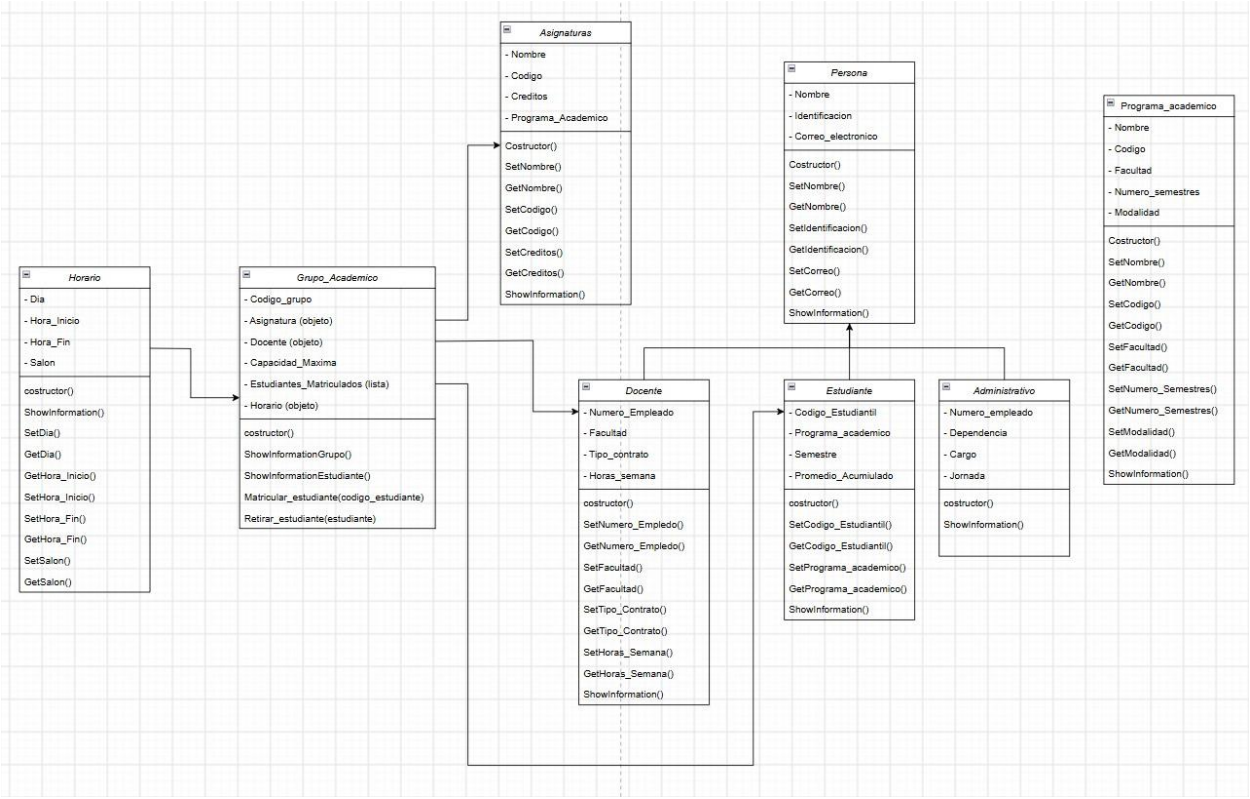
El archivo main.py demuestra cada punto solicitado en las pruebas mínimas de la actividad:

- Que no se puede crear una Persona directamente (se captura el TypeError).
- La creación de programas, asignaturas, docentes, estudiantes y horarios, con sus validaciones.
- La creación de dos grupos académicos (G01 y G02).
- La matrícula correcta de estudiantes en G01.
- El intento de matricular un estudiante duplicado (EstudianteYaMatriculadoError).

- El intento de superar la capacidad del grupo (GrupoLlenoError). Para esta prueba se creó un estudiante adicional con datos válidos, porque uno de los estudiantes de prueba se había creado a propósito con datos inválidos.
- El retiro correcto de un estudiante y el intento de retirar uno que no está matriculado (EstudianteNoMatriculadoError).
- La búsqueda de una asignatura y de un programa inexistentes (AsignaturaNoEncontradaError y ProgramaNoEncontradoError).
- La continuidad del programa después de cada excepción: la ejecución llega hasta el mensaje final y el recorrido polimórfico de AC1 sigue produciendo el mismo resultado.

Después de cada cambio se ejecutó main.py completo y se comparó la salida con el resultado esperado.

Diagrama de clases



Conclusiones

- La conversión de Persona en clase abstracta refleja mejor el problema real; en la universidad toda persona tiene un rol, y el sistema ahora lo hace cumplir,

impidiendo instancias genéricas y obligando a cada rol a definir su propia actividad principal.

- La separación entre asociación, agregación, composición y herencia permitió modelar correctamente la oferta académica. La herencia se reservó para relaciones “es un” (Estudiante, Docente y Administrativo frente a Persona), y la composición para relaciones “tiene un” (GrupoAcademico frente a Horario).
- El polimorfismo desarrollado en la AC1 se mantuvo intacto y se reforzó con el contrato que impone el método abstracto.
- Trabajar sobre el mismo proyecto y repositorio mostró la importancia de un diseño extensible: La AC2 se logró agregando clases y modificando solo lo necesario, sin rehacer lo anterior.
- Como mejora futura, el sistema podría validar cruces de horario entre grupos de un mismo docente o estudiante, y persistir la información en archivos o en una base de datos.
- Las excepciones personalizadas reemplazaron los mensajes sueltos por un mecanismo que interrumpe la operación errónea, informa con claridad y deja el programa en un estado consistente.

Declaración de uso de herramientas de inteligencia artificial

En cumplimiento de lo exigido por la actividad, el equipo declara sobre el uso de herramientas de inteligencia artificial que se utilizó Claude (Anthropic) como apoyo durante el desarrollo de esta actividad, para:

- Comprender conceptos (clases abstractas con ABC, composición frente a herencia y excepciones personalizadas).
- Proponer la estructura de las clases nuevas y generar fragmentos de código de Horario, GrupoAcademico, las excepciones y RegistroAcademico.
- Revisar errores y resultados de ejecución, y apoyar la redacción del README y de la estructura de este documento.

Las propuestas se revisaron y se modificaron o corrigieron en varios puntos:

- La primera versión de Horario imprimía directamente en ShowInformation(). Se cambió para que devolviera un texto, igual que el resto de las clases del proyecto, y los mensajes de error se ajustaron al formato ya usado.

- Los parámetros de Matricular_estudiante() y Retirar_estudiante() estaban invertidos en el diagrama de clases inicial. Se corrigieron de acuerdo con la guía y se actualizó el diagrama.
- Se corrigió el orden de las pruebas en main.py, porque un bloque usaba objetos que todavía no se habían creado.
- Se corrigió el nombre del archivo de excepciones (excepciones.py) para que coincidiera con los imports, y se adaptaron los nombres de clases y métodos reales del proyecto (por ejemplo, la clase asignaturas y el método GetCodigoEstudiantil()).

El código se probó ejecutando main.py tras cada paso. El uso de estas herramientas no sustituye la comprensión del trabajo; los integrantes del grupo pueden justificar cada clase, método y decisión descritos en este documento.